

Student Grade Management System

Complete Project Documentation & Technical Specification

1. Introduction

Student Grade Management System is a simple desktop application built using Python Programming Language. The software is designed to help carry out easy tasks such as adding, viewing, searching, editing, and deleting student records. The student's percentage and grade are also computed automatically by this system.

The application is written using Tkinter and JSON libraries which makes it lightweight and easy to understand. The main aim of developing this project is to demonstrate the working of various Python concepts like functions, if-elif-else conditions, lists, dictionaries, files, validations, and graphical user interface.

2. Problem Statement

Keeping track of students' marks and grades manually can be a tedious task and might lead to errors. Manually calculating percentages and maintaining student records may be time-consuming and challenging. Searching, editing, or deleting specific information from a student's record might be difficult.

Therefore, developing a computer software to carry out these tasks automatically is the main problem to be addressed by the Student Grade Management System.

3. Objectives

After analyzing the problem statement, the following objectives have been identified:

- To develop a simple Python application to demonstrate student records management.
- To add, store, compute percentages, and assign grades automatically.
- To search, edit, and delete student information efficiently while preventing roll number duplication and invalid values.
- To store records permanently via JSON files and provide a performance dashboard.

4. Technologies Used

Technology	Purpose
Python	Primary programming language offering simple and readable syntax.
Tkinter	Standard library for building GUI widgets (buttons, labels, tables).
JSON	Lightweight storage format for structured student dictionaries.
OS Module	Checking and working directly with local data files.
GitHub	Hosting and managing project source control repository.

5. System Features

5.1 Add Student: Users input student name, roll number, course, total marks, and marks obtained. Percentages and grades are auto-computed.

5.2 Percentage & Grade Calculation: Computes percentage via $(\text{Marks Obtained} / \text{Total Marks}) * 100$ and maps ranges (e.g., 90%+ is A+, 80-89.99% is A) to assign grades.

5.3 Search, Edit & Delete: Fast lookup across fields, full record updates, and safe database removal.

5.4 Dashboard & Persistence: Displays total students, average/highest percentages, and pass metrics, backed by persistent JSON storage.

6. System Requirements

6.1 Hardware Requirements

- Computer / Laptop with standard processor
- 4 GB RAM recommended
- Sufficient local disk storage for application and JSON files
- Standard keyboard and mouse

6.2 Software Requirements

- Operating System: Windows / Linux / macOS
- Python 3.x with standard Tkinter library
- Any standard Python IDE or code editor

7. Working of the System

1. **Launching:** User launches the desktop Python application.
2. **Loading Data:** Program loads existing records from the JSON data file.
3. **Data Entry & Validation:** User inputs student details; validations check for empty fields, correct numeric data types, positive marks, and unique roll numbers.
4. **Computation & Storage:** Percentage and grade are calculated, record is saved to JSON, and the table updates instantly.

8. Calculating the Student's Grade

Percentage Scored	Grade Awarded
90% and above	A+
80% – 89.99%	A
70% – 79.99%	B
60% – 69.99%	C
50% – 59.99%	D
Below 50%	F

9. Data Storage

Records are stored in `students_data.json` using Python dictionaries:

```
{ "name": "Example Student", "roll": "101", "course": "Computer Science",  
  "total_marks": 100, "marks_obtained": 85, "percentage": 85.0, "grade": "A" }
```

10. Validation and Error Handling

- **Empty Fields:** Mandates filling out name, course, and roll number.
- **Invalid Marks:** Restricts numeric fields to valid numbers only, preventing negative values or marks exceeding totals.
- **Duplicate Roll Numbers:** Rejects duplicate entries with an alert message: *"This roll number already exists"*.

11. User Interface Structure

The Tkinter graphical interface is split into clear functional blocks:

- **Header:** Displays application title banner.
- **Dashboard:** Summarizes total students, average percentage, highest score, and pass rates.
- **Student Details Form:** Input fields for name, roll number, course, and marks.
- **Student Table & Actions:** Tabular display paired with action buttons for saving, editing, deleting, and searching records.

12. Testing & Test Cases

Test Case	Expected Result	Status
Adding valid student record	Student successfully added and displayed	Passed
Leaving empty fields blank	Warning message displayed	Passed
Entering invalid marks	Error/warning message displayed	Passed
Adding duplicate roll number	Duplicate roll number rejected	Passed
Editing existing record	Student record successfully edited	Passed
Deleting record & persistence	Record removed / data reloaded correctly	Passed

13. Advantages and Limitations

Advantages

- Simple, elegant GUI with automated percentage and grade computations.
- Efficient search, update, and persistent local storage without heavy databases.

Limitations

- Designed for single-computer local use without multi-user login levels or cloud sync.

14. Future Scope

- **Login System & Roles:** Restrict access for authorized users.
- **Advanced Features:** Subject-wise marks, semester tracking, automated report cards, PDF/Excel export, and SQLite database migration.

15. Conclusion

The Student Grade Management System successfully demonstrates key Python programming concepts such as functions, control flow, data structures, and graphical interfaces. It offers a practical desktop utility for managing academic records reliably and efficiently.